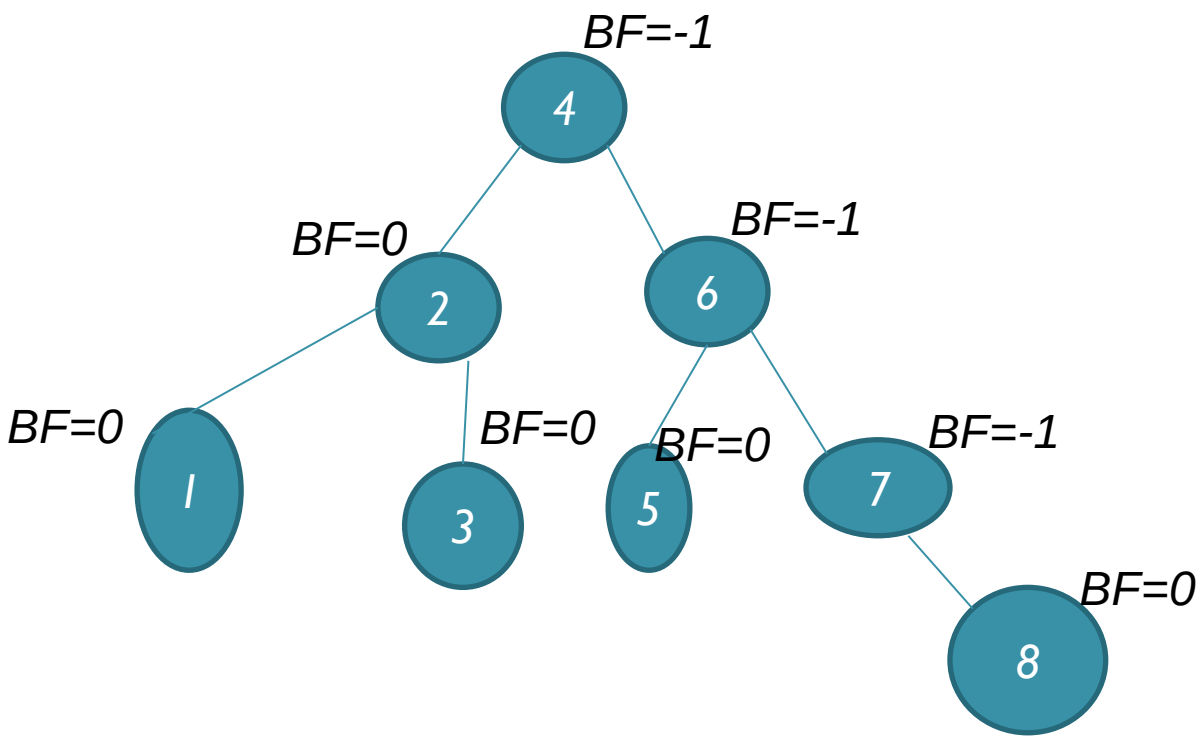
AVL Trees

- AVL tree is a height-balanced binary search tree
- It is also a binary search tree but it is a balanced tree.
- What is Balanced Tree?
  - A binary tree is said to be balanced if, the difference between the heights of left and right subtrees of every node in the tree is either -1, 0 or +1
- In an AVL tree, every node maintains an extra information known as balance factor

Construction of AVL Tree (1 to 8)

Example:



TREE is balanced

Node Structure

```
Struct Node
{
        int data;
        struct Node *left;
        struct Node *right;
        int height;
};
```

LEFT	DATA	RIGHT	HEIGHT

NODE

Height of the Node

```
Height(Struct Node *Node)
{
        if(Node==NULL)
                return 0;
        return Node->Height;
}
```

Creation of Node

```
Create newNode(int value)
{
        Struct Node *newnode;
        newnode= malloc(sizeof(Struct  node);
        newnode->data=value;
        newnode->left=NULL;
        newnode->right=NULL;
        newnode->height=1;
return newNode;

}
```

NULL	10	NULL	1

newNODE

Balance Factor of the Node

```
Balance Factor(Struct Node *Node)
{
        if(Node==NULL)
                return 0;
        return height(Node->left)-height(Node->right) ;
}
```

Left Rotate

```
Struct Node *Left Rotate( Struct node *x)
{
  Struct Node *y= x->right;
  Struct Node *T= y->left;
  y->left =x;
  x->right=T;
  x->height= max(height(x->left), height(x->right))+1;
  y->height= max(height(y->left), height(y->right))+1;
  return y;
}
```

Right Rotate

```
Struct Node *Right Rotate( Struct node *y)
{
  Struct Node *x= y->left;
  Struct Node *T= x->right;
  x->right=y;
  y->left =T;
   x->height= max(height(x->left), height(x->right))+1;
   y->height= max(height(y->left), height(y->right))+1;
   return x;
}
```

Finding Max

```
Max( a,b)
{
        return(a>b)? a:b;
}
```

//Returns which is maximum among a and b

Insertion Operation

Node and value to be inserted is passed every time

```
insert(struct * node, int value)
{
        if(node == NULL)
                return newNode(value);
        if(value < node->data)
        {
        node->left = insert(node- >left, value);
        else if(value > node->data)
        node->right =insert(node->right, value);
        else
                return node
         }
```

// founds the correct position and inserts node

Insertion Operation

```
// Update the height & Balance Factor of each node
Node->height =max[height(node->left), height(node->right)]+1;
Balance = Balance Factor(node);
// ROTATIONS
if(balance>1 && value<node>left->value)// RR Rotation
        return rightrotate(node);
if(balance <-1&& value>node->right->value)// LL Rotation
        return leftrotate(node);
if(balance >1&& value>node->left->value)// LR Rotation
        node->left= leftrotate(node->left);
        return rightrotate(node);
if(balance <-1&& value<node->right->value)// RL Rotation
        node->right= rightrotate(node->right);
        return leftrotate(node);
return node;
}
```

```
deleteNode(struct node* root, int value)
{
    if (root == NULL) // value not found
    return root;
    // key less than value of root then it lies in left subtree
    if (value< root->data)
        root->left = deleteNode(root->left, value);
    // key greater than value of root then it lies in right subtree
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
    // key is same as value of root, node be deleted is found
    else
    {
```

```
// node with only one child or no child
    if (root->left == NULL)
    {
            struct node* temp = root->right;
            free(root);
            return temp;
    }
    else if (root->right == NULL)
    {
            struct node* temp = root->left;
            free(root);
            return temp;
    }
```

```
  // node with two children, Get smallest in the right subtree)
    struct node* temp = minValueNode(root->right);
    root->value = temp->value;
    // Delete the node
    root->right = deleteNode(root->right, temp->value);
 }
    return root;
}
// Update the height & Balance Factor of each node
```

```
root->height =max[height(root->left), height(root->right)]+1;
Balance = Balance Factor(root);
// ROTATIONS
if(balance>1 && value<root->left->value)// RR Rotation
        return rightrotate(root);
if(balance <-1&& value>root->right->value)// LL Rotation
        return leftrotate(root);
if(balance >1&& value>root->left->value)// LR Rotation
        root->left= leftrotate(rotate->left);
        return rightrotate(root);
if(balance <-1&& value<root->right->value)// RL Rotation
        root->right= rightrotate(root->right);
        return leftrotate(root);
return root;
}
```